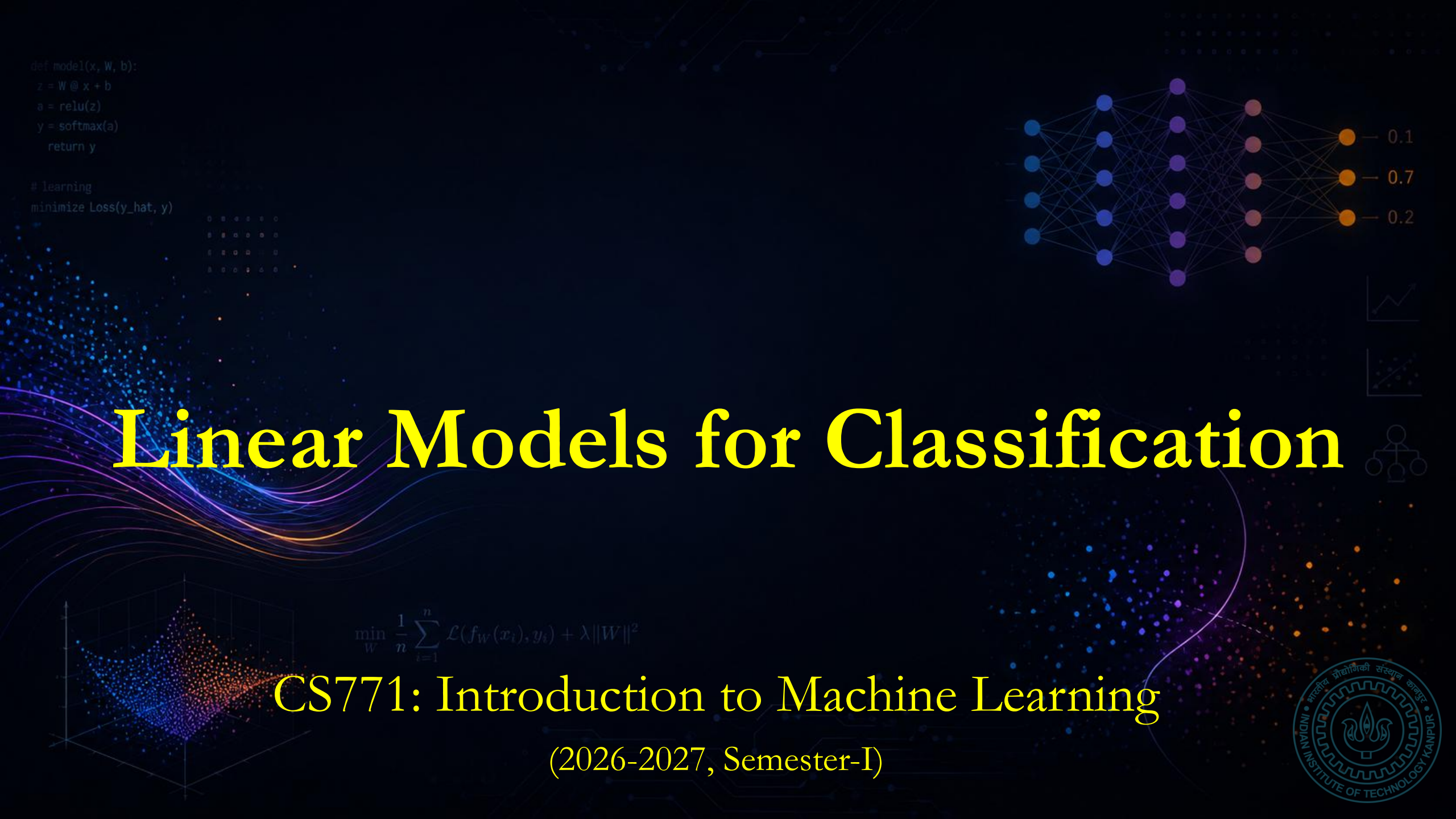


Linear Models for Classification

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

Learning Linear Models for Binary Classification

Binary classification: label $y_n \in \{-1, +1\}$ or $y_n \in \{0,1\}$

A linear model for binary classification does the following

1

Computes a score for input $\boldsymbol{x}_n$

$$s_n = \boldsymbol{w}^\top \boldsymbol{x}_n + b$$

2

Converts the score to binary prediction

$$\hat{y}_n = \text{sign}(s_n)$$

or

$$\hat{y}_n = \mathbb{I}(s_n > 0)$$

Optional **3**

Done only if we also need estimates of class probabilities for $\boldsymbol{x}_n$

A mapping function $g\colon \mathbb{R} \to (0,1)$ maps the score to an estimated probability of label being +1.

$$\hat{p}_n = g(s_n)$$

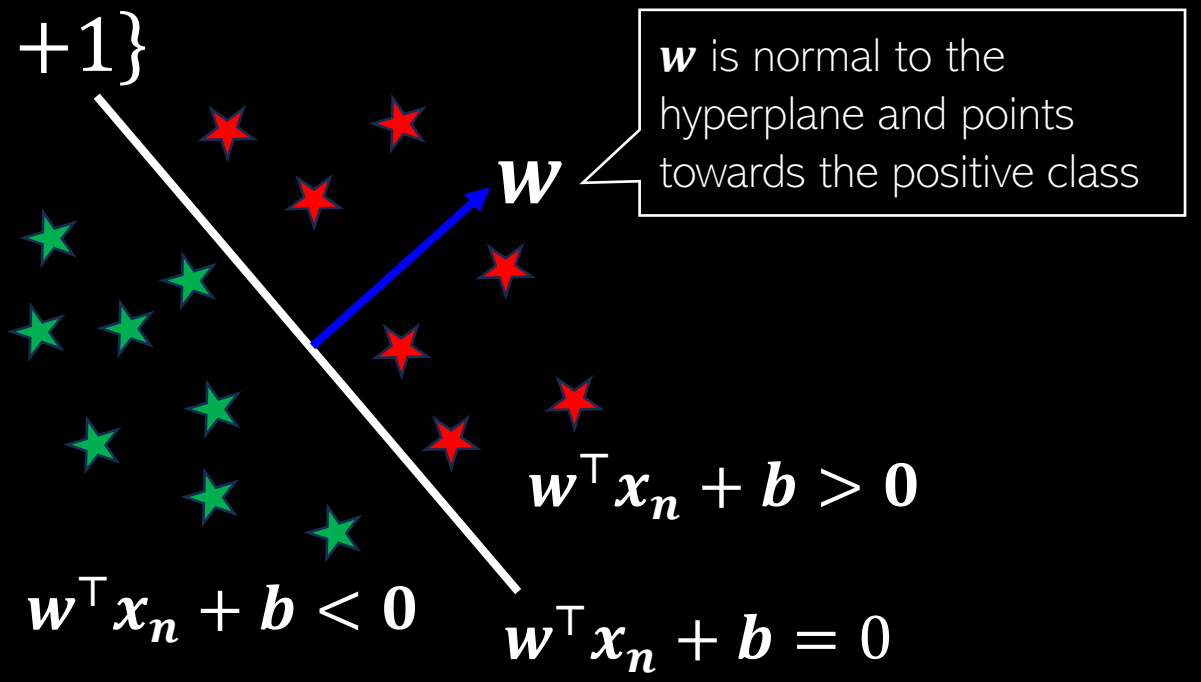


Decision Boundary of Linear Classifiers

Assume input $\boldsymbol{x}_n$ with labels $y_n \in \{-1, +1\}$

Score: $s_n = \boldsymbol{w}^\top x_n + b$

Prediction: $\hat{y}_n = \text{sign}[s_n]$



The decision boundary

$$\mathcal{H} = \{\boldsymbol{x} : \boldsymbol{w}^\top x + b = 0\}$$

- Line in two dimensions
- Plane in three dimensions
- "Hyperplane" in higher dimensions

Collectively, $(\boldsymbol{w}, b)$ determine the orientation and position of the decision boundary

Linear Classifiers Learn Nonlinear Classification?

Yes, definitely possible using a suitable nonlinear feature transformation $\boldsymbol{\phi}(\boldsymbol{x}_n)$ and a linear classifier on those features

Can be hand-crafted or learned

$$\mathcal{H} = \{\boldsymbol{x} : \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}) + b = 0\}$$

Decision boundary is a hyperplane in $\boldsymbol{\phi}$ feature space but can be nonlinear in original input space

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \boldsymbol{\phi}(\boldsymbol{x}) = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix}$$

A hand-crafted transformation

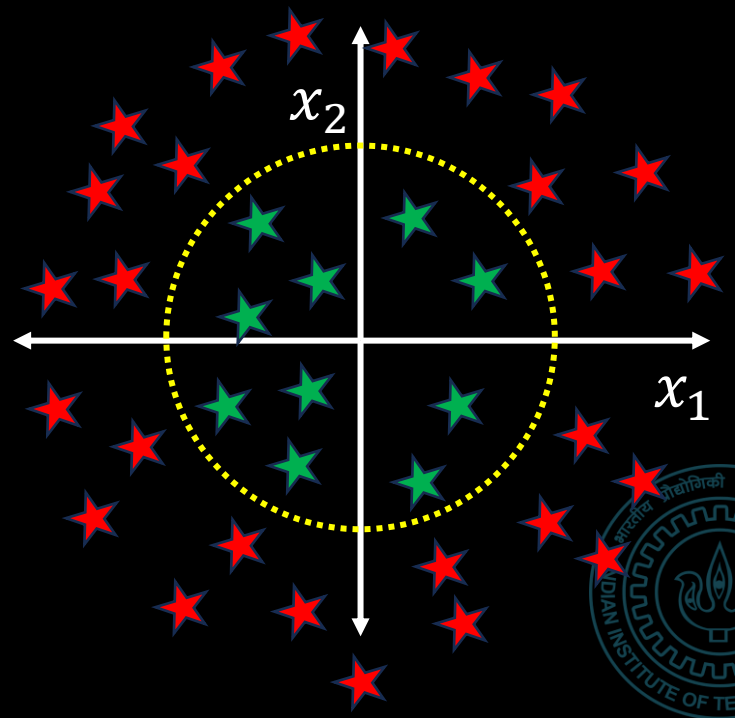
$$\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}) + b = 0$$

Linear decision boundary (defined by $\boldsymbol{w}, b$) in $\boldsymbol{\phi}$ space

$$\boldsymbol{w} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad b = -1$$

Nonlinear decision boundary in the original feature space

$$x_1^2 + x_2^2 - 1 = 0$$



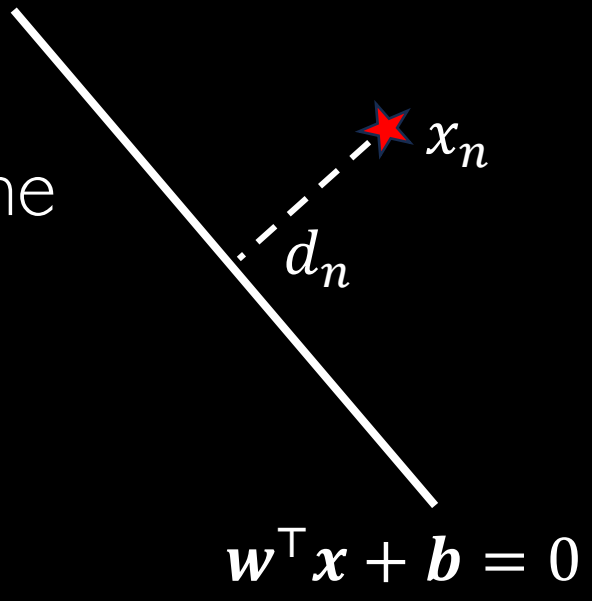
Binary Classification: Scores and Margins

Some quantities related to score $s_n = \boldsymbol{w}^\top \boldsymbol{x}_n + b$:

- Signed Euclidean distance of $\boldsymbol{x}_n$ from the hyperplane

Scaling $(\boldsymbol{w}, b)$ by a positive constant (say $(2\boldsymbol{w}, 2b)$) would affect s_n but not affect d_n

$$d_n = \frac{\boldsymbol{w}^\top \boldsymbol{x}_n + b}{\|\boldsymbol{w}\|} \propto s_n$$



- Functional margin and geometric margin of $\boldsymbol{x}_n$ whose true label is $y_n \in \{-1, +1\}$

Functional margin

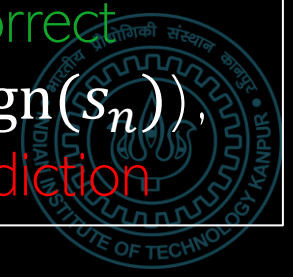
$$m_n = y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b) = y_n s_n$$

Geometric margin

Invariant to positive scaling of $(\boldsymbol{w}, b)$

$$\gamma_n = \frac{y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b)}{\|\boldsymbol{w}\|}$$

Both functional and geometric margins are signed quantities. They are > 0 for correct prediction ($y_n = \text{sign}(s_n)$), and < 0 for misprediction

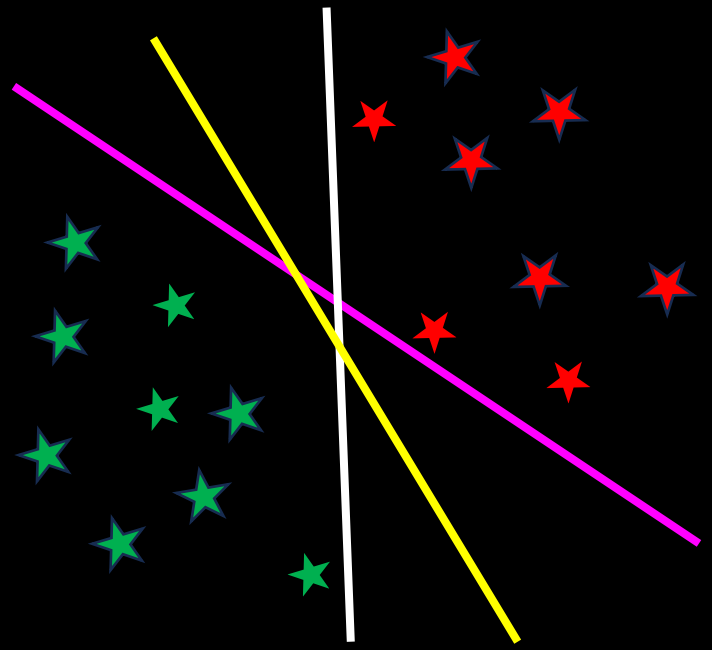


Large Margins are Important

Often many separators can separate the training data perfectly

$$\gamma = \min_n \frac{y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b)}{\|\boldsymbol{w}\|}$$

Margin of a hyperplane separator $(\boldsymbol{w}, b)$ is the minimum geometric margin over the training set



Larger margins can favor more robust separators and are often associated with better generalization.

The large-margin principle extends to nonlinear classifiers too

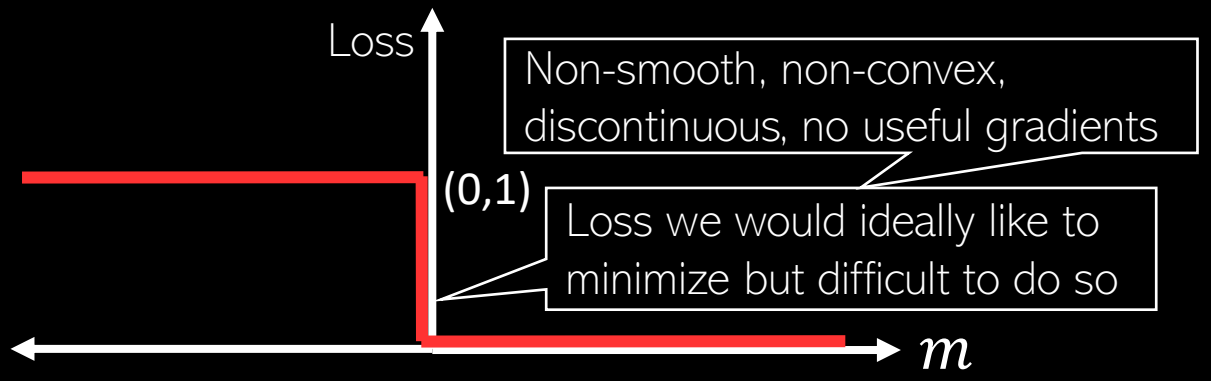
Among perfect separators, prefer one that stays far from training examples

We would like to learn the hyperplane with maximum possible margin γ.
Maximum margin classifiers like SVM do precisely this (will revisit this later)

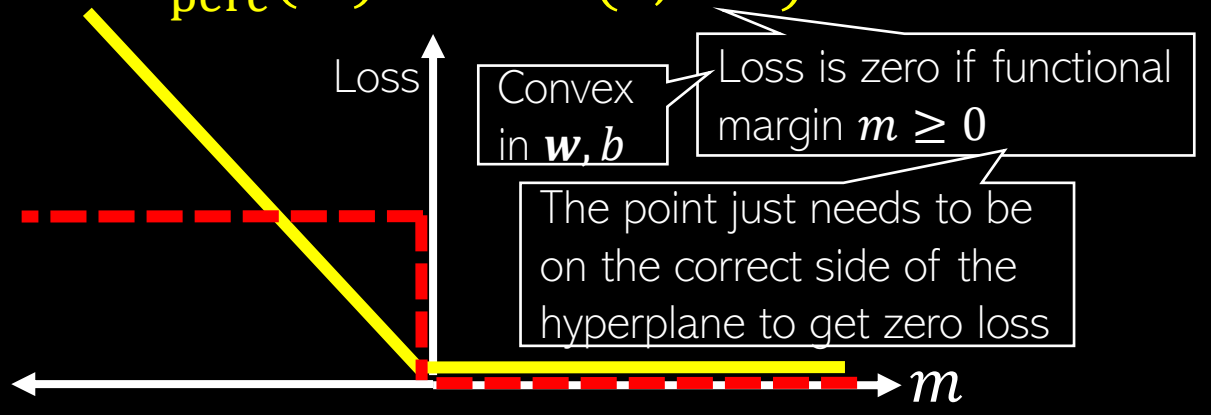
Popular Loss Functions for Binary Classification

Can define classification losses as a function of margin $m = y(\boldsymbol{w}^\top \boldsymbol{x} + b)$

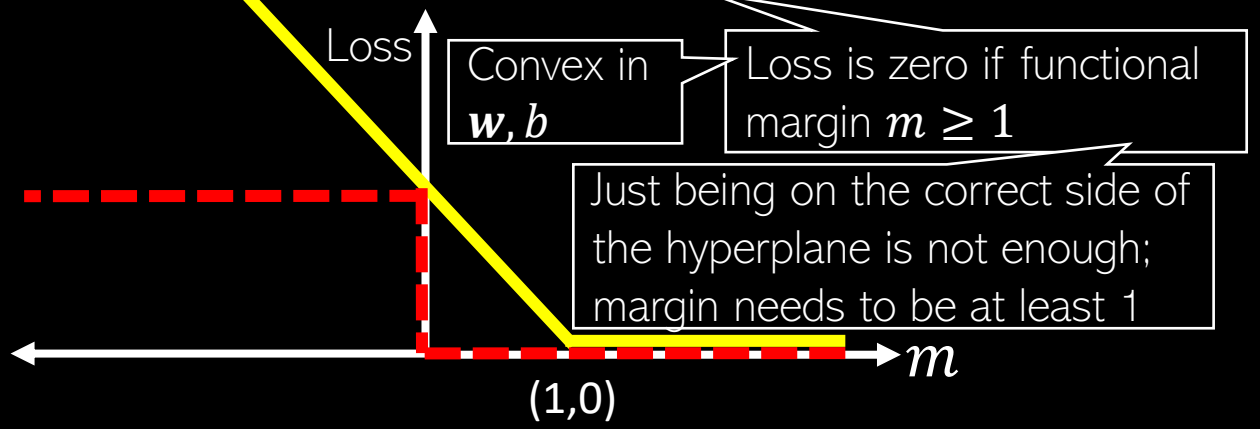
$\ell_{0-1}(m) = \mathbb{I}[m \leq 0]$



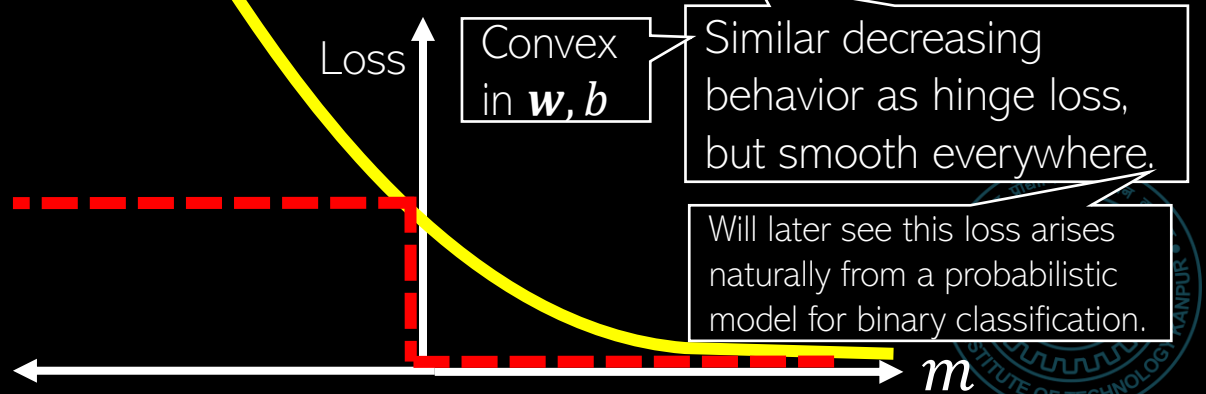
$\ell_{\text{perc}}(m) = \max(0, -m)$



$\ell_{\text{hinge}}(m) = \max(0, 1 - m)$



$\ell_{\log}(m) = \log(1 + \exp(-m))$



Objectives for Some Popular Binary Classifiers

The Perceptron classifier

$$\mathcal{L}(\boldsymbol{w}, b) = \frac{1}{N}\sum_{n=1}^{N} \max(0, -y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b))$$

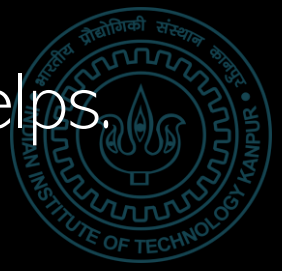
Support Vector Machine

$$\mathcal{L}(\boldsymbol{w}, b) = \frac{1}{N}\sum_{n=1}^{N} \max(0, 1 - y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b)) + \lambda \boldsymbol{w}^\top \boldsymbol{w}$$

Logistic Regression with ℓ_2 regularizer

$$\mathcal{L}(\boldsymbol{w}, b) = \frac{1}{N}\sum_{n=1}^{N} \log(1 + \exp\big(-y_n(\boldsymbol{w}^\top \boldsymbol{x}_n + b)\big) + \lambda \boldsymbol{w}^\top \boldsymbol{w}$$

These can be optimized using GD/SGD or their variants. Convexity helps.



From Loss to Learning: What Does GD Do?

Consider GD/SGD on $\ell_{\log}(m) = \log(1 + \exp(-m))$ where $m = y(\boldsymbol{w}^\top \boldsymbol{x} + b)$

Gradient for each example $(\boldsymbol{x}_n, y_n)$

$$\nabla_w \ell_{\log}(m_n) = \frac{\partial \ell_{\log}}{\partial m_n}\frac{\partial m_n}{\partial w} = -\frac{1}{1+\exp(m_n)} y_n \boldsymbol{x}_n = -\sigma(-m_n) y_n \boldsymbol{x}_n$$

Exercise: Try deriving SGD update for the Perceptron loss and Hinge loss

SGD update for $\boldsymbol{w}$ using the example $(\boldsymbol{x}_n, y_n)$:

$$\boldsymbol{w}^{(\text{new})} = \boldsymbol{w}^{(\text{old})} + \eta\sigma(-m_n) y_n \boldsymbol{x}_n$$

where $\sigma(z) = \frac{1}{1+\exp(-z)}$

$\sigma(-m_n)$ is the strength of correction (where $m_n = y_n(\boldsymbol{w}^{(old)\top} \boldsymbol{x}_n + b)$)

- Large negative margin $m_n \rightarrow$ large correction to $\boldsymbol{w}^{(\text{old})}$
- Near the boundary ($m_n \approx 0$) $\rightarrow$ moderate correction
- Large positive margin $\rightarrow$ negligible/tiny correction

Exercise: Show that the margin on $(\boldsymbol{x}_n, y_n)$ becomes more positive (or less negative) increases after update

Binary Classification: Estimate of Class Probability

Can map score $s_n = \boldsymbol{w}^\top \boldsymbol{x}_n + b$ to an *estimated* probability that $y_n = +1$

$$\hat{p}_n = g(s_n)$$

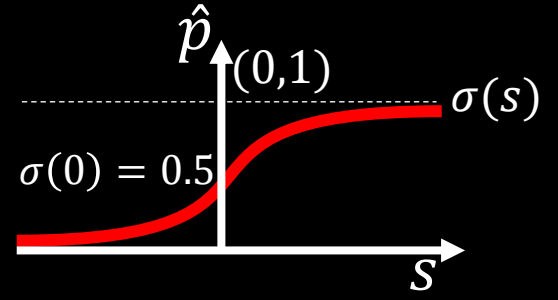
Typically monotonically increasing with score s

$g(s) \to 1$ for Large positive s, $g(s) \to 0$ for large negative s

Here $g(s)$ is a suitable score-to-probability mapping function $g: \mathbb{R} \to (0,1)$

For a probabilistic classifier, $g(s)$ is defined by the model itself, e.g., for logistic regression, g is the sigmoid function

$$\hat{p}_n = g(s_n) = \text{sigmoid}(s_n) = \frac{1}{1 + \exp(-s_n)}$$



For non-probabilistic classifiers, calibration methods can be used to map scores to estimated probabilities (e.g., Platt scaling for SVMs)